

Aufgabenblatt 4

Übung zu Einführung in die Bildverarbeitung

Hanna Beitz, Malin Haas, Lasse Huber-Saffer, Tim Radke,
Teodora Rogozenco, Jannis Scheff, Christian Wilms
SoSe 2026

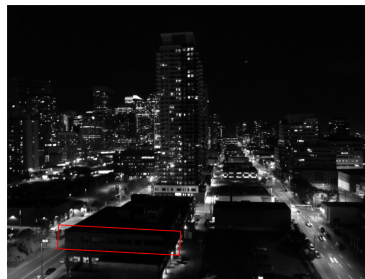
Ausgabe: 8. Mai 2026 - **Abgabe bis: 22. Mai 2026, 10:00 per Moodle!**

Lernziele

- Effekt verschiedener Intensitätstransformationen vorhersagen
- Verständnis zum Aufbau von Intensitätstransformationen
- Zusammenspiel von geometrischen Transformationen verstehen
- Umsetzung und einfache Interpolationsmechanismen vertiefen



(a)



(b)



(c)

Abbildung 1: Bilder für die Bildverbesserung in Aufgabe 1.

Aufgabe 1 — Bildverbesserung - 8+8+9=25 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `matplotlib`, `math`

Abbildung 1 zeigt drei Bilder, die im Rahmen der folgenden Teilaufgaben mit Hilfe von jeweils einer Intensitätstransformation so verbessert werden sollen, dass eine vorgegebene Information einfacher aus den Bildern ausgelesen werden kann. Überlegt euch dazu zunächst welche Intensitätstransformation je Teilaufgabe passend erscheint (mit kurzer Begründung) und implementiert diese jeweils in Python. Die in Abbildung 1 dargestellten Bilder findet ihr im Moodle als **bildverbesserung.zip**.

Hinweis 1: Für manche Bilder kann es sich anbieten, die Pixelwerte vom Wertebereich $0, \dots, 255$ auf den Bereich $0, \dots, 1$ zu ändern.

Hinweis 2: Die Ergebnisse werden nicht perfekt werden!

Hinweis 3: Es sollen stets alles Pixel eines Bildes mit der gleichen Funktion verändert werden.

1. Erhöht den Kontrast des Bildes in Abbildung 1a und zählt, wie viele Kondensstreifen es im Himmel gibt.
2. Verbessert den Sichtbarkeit des Gebäudes unten links in Abbildung 1b und ermittelt, wie viele Fenster die vordere Front des Gebäudes im obersten Stockwerk hat. Der Bildbereich ist in Abbildung 1b grob markiert.
3. Verändert das Bild in Abbildung 1c so, dass in etwa der Bereich der Autobahn in der Bildmitte weiß ist und der Rest des Bildes größtenteils seine Graufärbung behält.

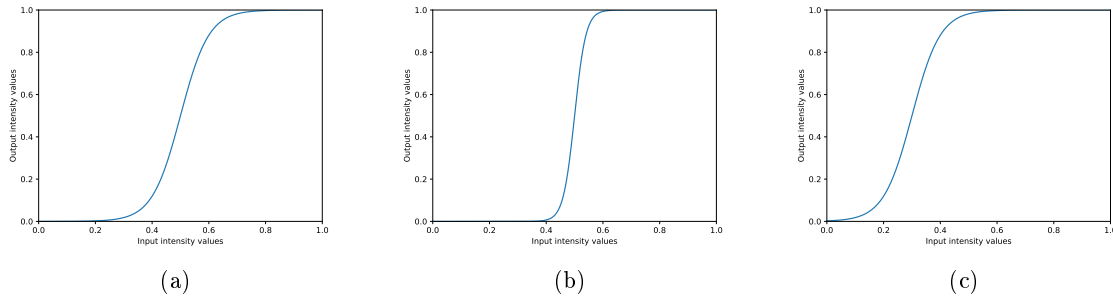


Abbildung 2: Variationen einer Intensitätstransformation zur Kontrastverbesserung.

Aufgabe 2 — Entwurf von Intensitätstransformationen - 20+5+5=30 Punkte - Theorieaufgabe
In der Vorlesung wurde eine Intensitätstransformation zur Kontrastverbesserung vorgestellt, die etwa die Form des Graphen in Abbildung 2a hat. In dieser Aufgabe soll eine mathematische Funktion dazu entwickelt werden, die überdies auch die weiteren in Abbildung 2 gezeigten Graphen durch geeignete Parametrisierung annehmen kann. Geht zur Vereinfachung für diese Aufgabe davon aus, dass die Helligkeitswerte zwischen 0 und 1 liegen, also $0, \frac{1}{255}, \frac{2}{255}, \dots, \frac{255}{255}$ sind.

1. Schlagt eine Funktion vor, die etwa eine Intensitätstransformation mit dem in Abbildung 2a gezeigten Graphen darstellt.
2. Wie lässt sich nun die Steigung verändern, wie in Abbildung 2b gezeigt? Wofür ist das im Rahmen der Bildverbesserung nützlich?
3. Wie lässt sich nun der Wendepunkt verschieben, wie in Abbildung 2c gezeigt? Wofür ist das im Rahmen der Bildverbesserung nützlich?

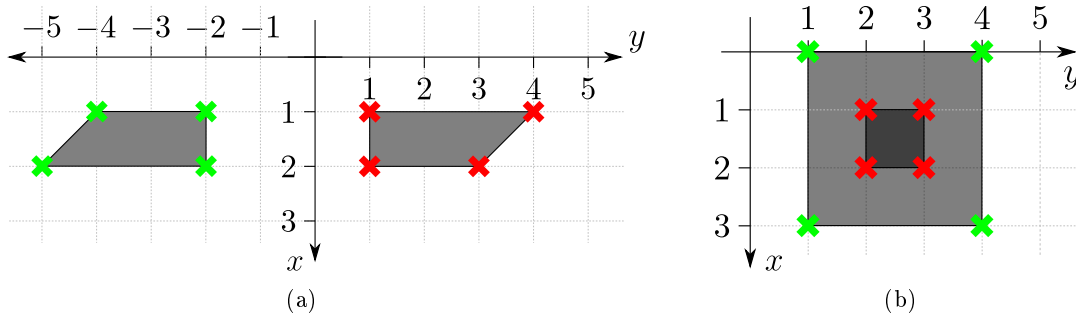


Abbildung 3: Zwei Szenarien, in denen jeweils eine Transformationsmatrix auf die Eckpunkte eines Vierecks (rote Kreuze) angewendet wurde, um ein neues Viereck zu erzeugen (grüne Kreuze).

Aufgabe 3 — Geometrische Transformationen - 5+10+5+5=25 Punkte - Theorieaufgabe
Diese Aufgabe widmet sich geometrischen Transformationen.

1. Gebt zu den zwei Szenarien in Abbildung 3 die jeweilige Transformationsmatrix an. In den zwei Szenarien sollen die Transformationen jeweils auf die Eckpunkte der Vierecke mit den roten Kreuzen angewendet werden, sodass jeweils das veränderte Vierecke (grüne Kreuze) entsteht. Bei Sequenzen von Transformationen sollen die einzelnen Transformationsmatrizen und die Berechnung der finalen, kombinierten Transformationsmatrix gegeben werden.

Tipp: Wendet einmal eure erzeugte Transformationsmatrizen auf die entsprechenden Punkte an und betrachtet das Ergebnis. Stimmt es?

2. Beweist oder widerlegt mit einem Gegenbeispiel die Kommutativität folgender geometrischer Transformationen bei der Anwendung auf Punkte $\begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$.

- a) Scherung und Translation
- b) Uniforme Skalierung und uniforme Skalierung mit unterschiedlichen Faktoren je Skalierung.

Aufgabe 4 — Transformationsrätsel - 20 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.transform`, `itertools`, `matplotlib`

Das Bild `brett.jpg` im Moodle zeigt eine verzerrte Aufnahme eines eigentlich rechteckigen Bretts. Wendet nun geometrische Transformationen so auf das Bild an, dass das Brett wieder näherungsweise die Form eines Rechtecks hat. Überlegt euch zunächst, welche Transformationen ihr benötigt und implementiert eure Idee anschließend. Ihr dürft sowohl das Ausgangsbild als auch alle Zwischenergebnisse so zuschneiden, dass sie euch für die Anwendung der Transformationen passend erscheinen. Am Ende sollte jedoch wieder das ganze Brett samt Hintergrund zu sehen sein.

Hinweis 1: Wendet die Transformationen nicht unbedingt auf das ganze Bild an, sondern zerlegt es nacheinander in Teile, die unterschiedlich behandelt werden.

Hinweis 2: Das Ergebnis muss nicht perfekt sein und im Hintergrund dürfen auch Teile fehlen, also bspw. mit schwarzen Pixeln gefüllt sein.

Zusatzaufgabe 5 — Bildskalierung und Interpolation - $10+15+5=30$ Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `matplotlib`, `itertools`

Durch die geometrische Transformation von Bildern, bspw. die uniforme Skalierung um einen Faktor s , werden Pixelwerte an Koordinaten des Ursprungsbildes benötigt, die zwischen den eigentlichen Koordinaten liegen. So kann es etwa vorkommen, dass ein Wert an der Koordinate $(0.2, 0.8)$ benötigt wird. Um an diesen Stellen Werte zu erzeugen, wird eine Interpolation vorgenommen, wie in der Vorlesung diskutiert. Dies soll in dieser Aufgabe implementiert werden.

1. Schreibt eine Python-Funktion, die ein Bild um einen gegebenen Faktor, der nicht ganzzahlig sein muss, uniform skaliert. Implementiert dazu die nearest neighbor Interpolation. Testet nun eure Skalierung am Bild `tv.png` aus dem Moodle.
Hinweis: Rechnet die Koordinaten des Zielbildes in die Koordinaten des Ausgangsbildes zurück und interpoliert dort (inverse mapping).
2. Schreibt nun eine zweite Python-Funktion, welche die selbe Skalierung vornimmt, aber die bilineare Interpolation nutzt.
3. Werden durch das Skalieren von Bildern mit einem Faktor $s > 1$ Details des Fernsehers gewonnen? Was passiert, wenn ihr das Bild zunächst um die Hälfte verkleinert ($s = 0.5$) und anschließend wieder auf die vorherige Größe vergrößert ($s = 2$)? Ist das Ergebnis identisch zum Ursprungsbild?